

DMA Transfer

DMA Transfer

1. Learning Objectives
 - DMA Principle
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. ADC Parameter Configuration
 3. DMA Parameter Configuration
 4. Write Program
 5. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Understand basic DMA knowledge.
2. Learn the basic use of DMA and display the transferred ADC data on the serial assistant.

DMA Principle

DMA (Direct Memory Access) is a technology used in computer systems that allows peripherals to directly access memory without CPU involvement to complete data transfer. It enables efficient data exchange between peripherals (such as ADC, DAC, memory, etc.) and memory, reducing CPU burden and improving overall system performance.

Principle:

- **Trigger Data Transfer:** DMA operations are usually triggered by external devices (such as ADC, sensors) or internal events (such as timer overflow).
- **Initialize DMA Controller:** CPU configures parameters for DMA controller including source address, destination address, transfer data size, transfer direction, etc.
- **Direct Data Transfer:** Once the DMA controller is activated, it can directly transfer data from the source address (peripheral output buffer or memory) to the destination address (memory or peripheral) without CPU intervention.
- **Notify CPU After Transfer Complete:** After DMA completes data transfer, it can send an interrupt request to the CPU, telling the CPU that data is ready or transfer completed successfully.

2. Hardware Setup

The DMA in these devices supports the following important features:

- 12 independent DMA transfer channels
- 12 independent DMA transfer channels
- 6 full-featured channels (DMA0-DMA5), supporting repeated transfer mode
- 6 basic channels (DMA6-DMA11), supporting single transfer mode
- Configurable DMA channel priority
- Byte (8-bit), short word (16-bit), word (32-bit), long word (64-bit), and ultra-long word (128-bit) or mixed byte and word transfer capability
- Transfer counter block size supports transferring any type of data up to 64k
- Configurable DMA transfer trigger selection
- Active channel interrupt providing service for other channels
- Early interrupt generation for ping-pong buffer architecture
- Cascade channel when activity on another channel completes
- Stride mode supporting data reorganization, such as three-phase metering applications
- Collection mode

Table 8-2. DMA Features

Feature	FULL	BASIC
Channel#	0,1,2,3,4,5	6,7,8,9,10,11
Repeat Mode	Yes	-
Table & Fill Mode	Yes	-
Gather Mode	Yes	-
Pre-IRQ	Yes	-
Auto Enable	Yes	Yes
Long Long (128-bit) Transfer	Yes	Yes
Stride Mode	Yes	Yes
Cascading Channel Support	Yes	Yes

By consulting the data manual, only full-featured DMA channels support repeated transfer, early interrupt, and extended mode. Basic DMA channels support basic data transfer and interrupts, but are sufficient for simple data transfer requirements.

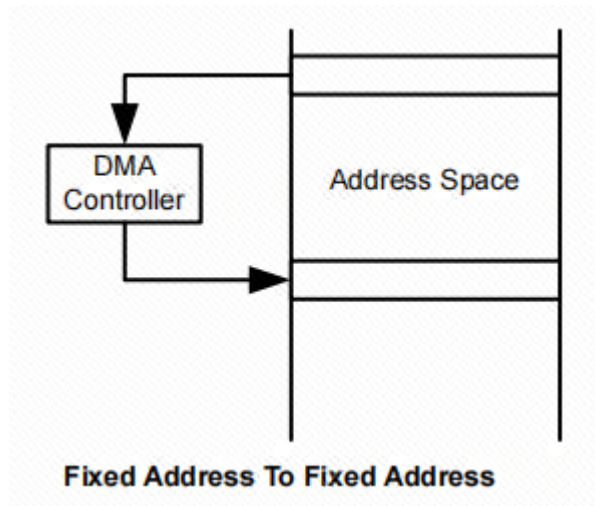
MSPM0G3519's DMA supports four transfer modes, each channel can be configured separately for its transfer mode. For example, channel 0 can be configured for single-word or single-byte transfer mode, channel 1 for block transfer mode, and channel 2 for repeated block transfer mode. Transfer modes are configured independently of addressing modes. Any addressing mode can be used with any transfer mode. Three types of data can be transferred, selected through .srcWidth and .destWidth control bits. Source and destination locations can be byte, short word, or word data. Transfers are also supported in byte-to-byte, short-word-to-short-word, word-to-word, or any combination. As follows:

- Single-word or single-byte transfer: Each transfer requires a separate trigger. DMA is automatically disabled when DMAxSZ transfer has been generated.
- Block transfer: An entire block will be transferred after one trigger. DMA is automatically disabled at the end of block transfer.
- Repeated single-word or single-byte transfer: Each transfer requires a separate trigger, DMA remains enabled.

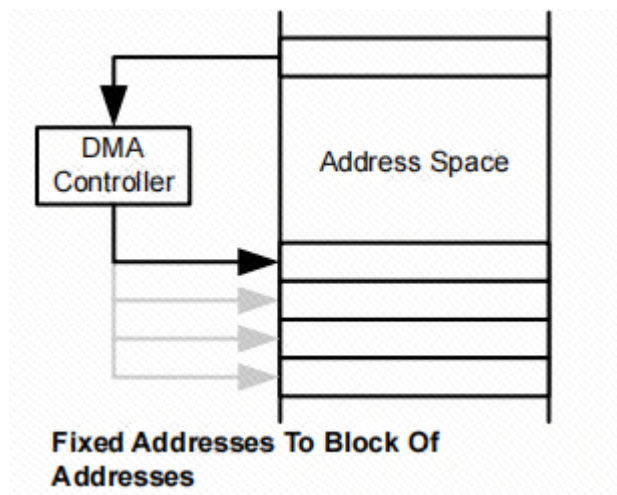
- Repeated block transfer: An entire block will be transferred after one trigger, DMA remains enabled.

Based on these modes, MSPM0G3519 provides 6 addressing modes:

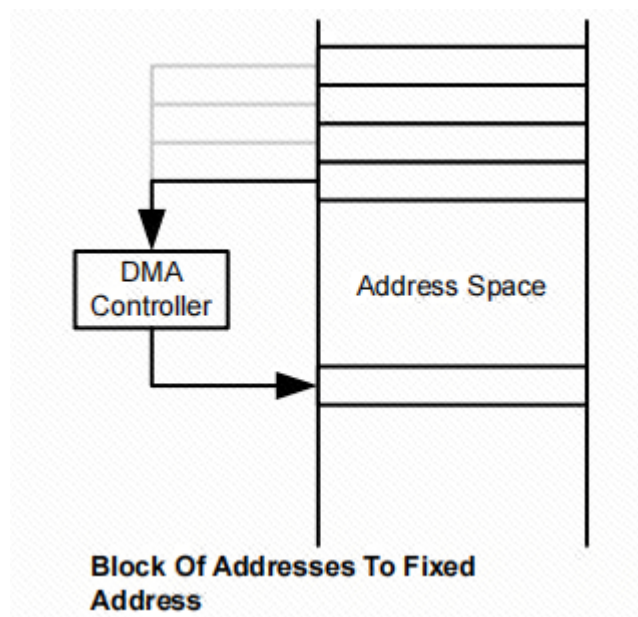
Fixed address to fixed address:



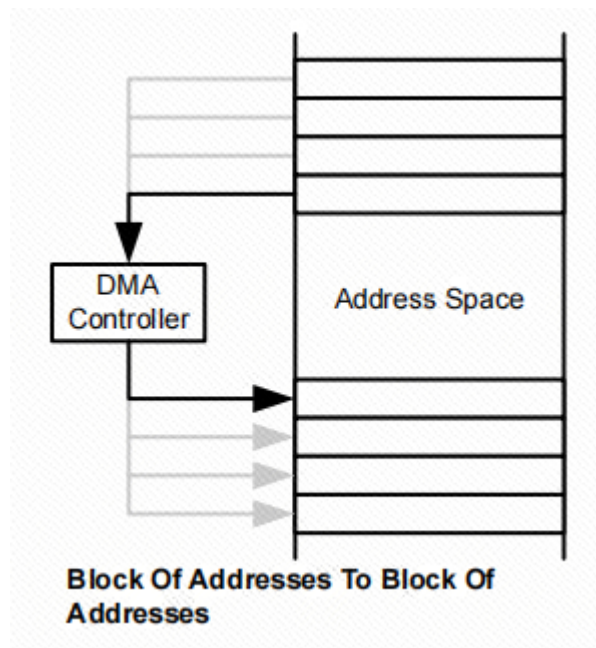
Fixed address to block address:



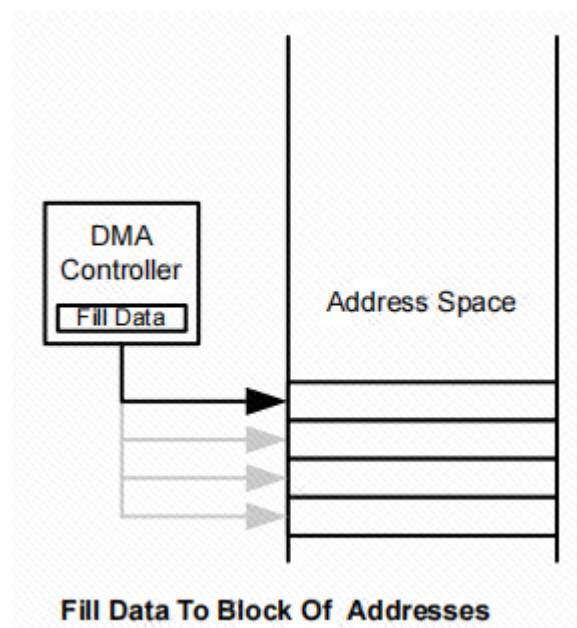
Block address to fixed address:



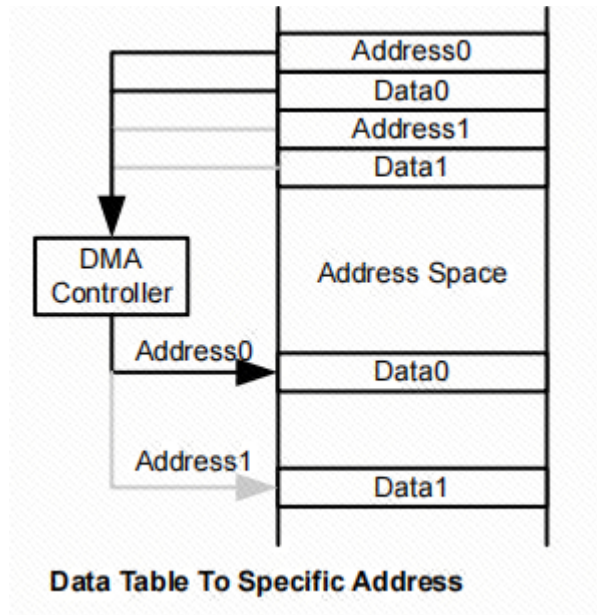
Block address to block address:



Fill data to block address:



Data table to specified address:

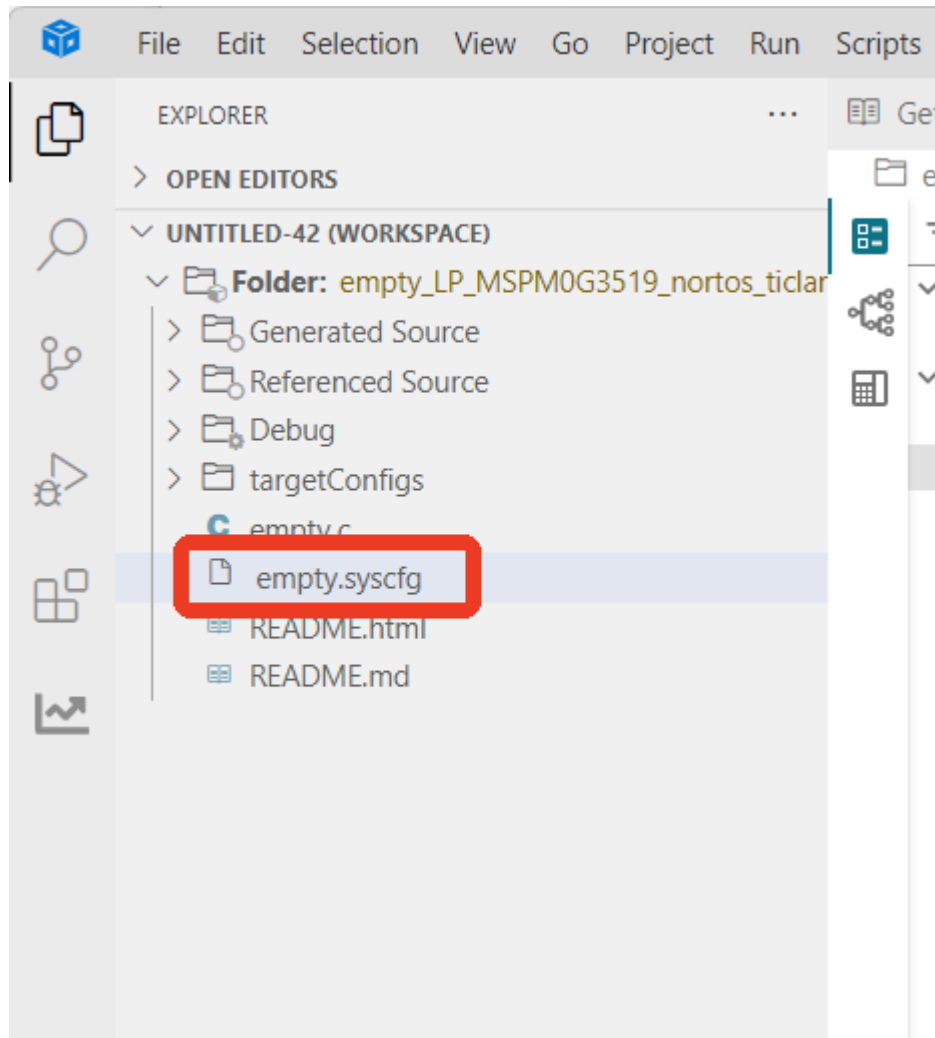


3. Experiment Steps

This case uses ADC to acquire PA27 pin voltage, and DMA transfers the ADC acquired data directly to a specified address.

1. Open SYSCONFIG Configuration Tool

This case uses the **ADC Acquisition** course as a template. Open the ADC project and open the .syscfg file.



2. ADC Parameter Configuration

The ADC configuration here is the same as in the **ADC Acquisition** case.

Configure ADC resolution and interrupts. This case does not use interrupts, so nothing is selected here.

Advanced Configuration



Total Conversion Frequency

9.78 kHz

i When Power Down Mode is set to Auto, ADC wakeup time may need to be considered in each sample window. Refer to the device specific data sheet for specifications on the ADC Wakeup Time.

Conversion Resolution

12-bits



Enable FIFO Mode

☐

Power Down Mode

Auto



Desired Sample Time 0

100 us

Actual Sample Time 0

100.00 μ s

Desired Sample Time 1

0 ms

Actual Sample Time 1

0.00 s

Interrupt Configuration



Enable Interrupts

None



3. DMA Parameter Configuration

DMA Configuration ^

Configure DMA Trigger☒

Enable DMA Trigger☒

DMA Samples Count1

Enable DMA TriggersMEM0 result loaded interrupt▼

DMA Channel ^

NameDMA_CH0

Channel ID0
Currently using a Full Channel.

Address ModeFixed addr. to Block addr.▼

Source LengthHalf Word (2 Bytes)▼

Destination LengthHalf Word (2 Bytes)▼

Destination Address DirectionIncrement▼

Configure Transfer Size☒

Transfer Size10

Transfer ModeRepeat Single▼

Source Address IncrementDo not change address after each transfer▼

Destination Address IncrementDo not change address after each transfer▼

Interrupt Configuration ^

Enable Channel Interrupt☐

Enable Early Channel Interrupt☐

Save the configuration in the .syscfg file using the shortcut key Ctrl+S.

4. Write Program

In the empty.c file, write the following code

```
#include "ti_msp_dl_config.h"
#include "stdio.h"

volatile uint16_t ADC_VALUE[20]; //The data collected by ADC is saved in the address

unsigned int adc_getValue(unsigned int number); //Read ADC data
void uart0_send_string(char* str); //Send string via serial port

int main(void)
{
    char output_buff[50] = {0};
    unsigned int adc_value = 0;
```

```

float voltage_value = 0;

SYSCFG_DL_init();

// Set the starting address of DMA transfer
DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC0->ULLMEM.MEMRES[0]);
// Set the destination address of DMA transfer
DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC_VALUE[0]);
// Enable DMA
DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
// Start ADC conversion
DL_ADC12_startConversion(ADC_VOLTAGE_INST);

uart0_send_string("adc+dma Demo start\r\n");
while (1)
{
    // Get ADC data
    adc_value = adc_getValue(10);
    sprintf(output_buff, "adc value:%d\r\n", adc_value);
    uart0_send_string(output_buff);

    // Convert the data collected by ADC into voltage
    voltage_value = adc_value/4095.0*3.3;
    sprintf(output_buff, "voltage value:%.2f\r\n", voltage_value);
    uart0_send_string(output_buff);

    delay_cycles(32000000);
}
}

// Read ADC data
unsigned int adc_getValue(unsigned int number)
{
    unsigned int gAdcResult = 0;
    unsigned char i = 0;

    // Collect multiple times and accumulate
    for( i = 0; i < number; i++ )
    {
        gAdcResult += ADC_VALUE[i];
    }
    // Mean Filter
    gAdcResult /= number;

    return gAdcResult;
}

// Send string via serial port
void uart0_send_string(char* str)
{
    //The current string address is not at the end and the string first address is not empty
    while(*str!=0&&str!=0)
    {

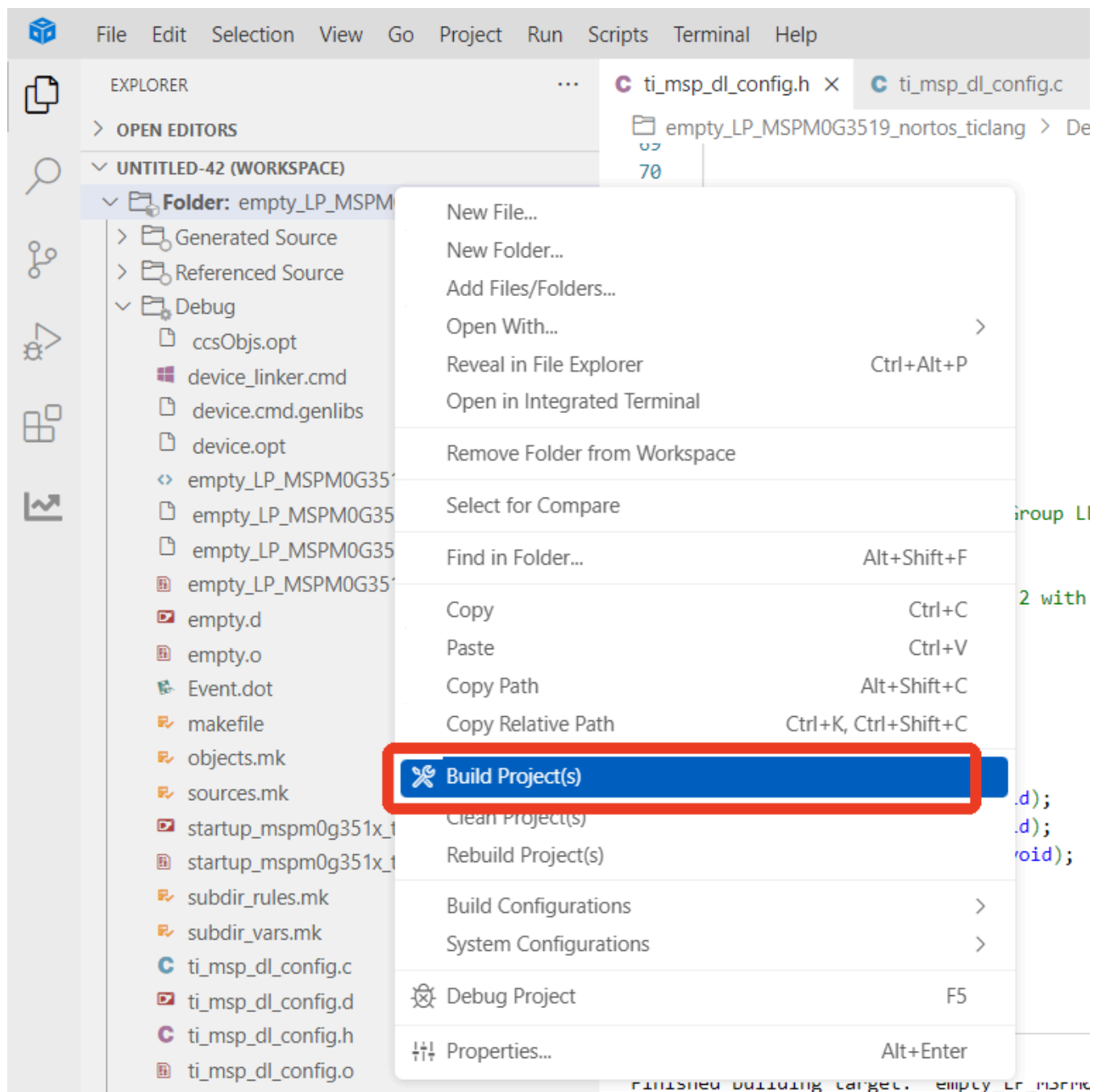
```

```

        // wait when serial port 0 is busy, and send the incoming characters when it is not
        busy
        while( DL_UART_isBusy(UART_0_INST) == true );
        // send the characters in the first address of the string, and increment the first
        address after sending.
        DL_UART_Main_transmitData(UART_0_INST, *str++);
    }
}

```

5. Compile



Compilation was successful. You can now download the program to the development board.

4. Program Analysis

- empty.c

```

60 //串口发送字符串 Send string via serial port
61 void uart0_send_string(char* str)
62 {
63     //当前字符串地址不在结尾 并且 字符串首地址不为空
64     //The current string address is not at the end and the string first address is not empty
65     while(*str!=0&&str!=0)
66     {
67         //当串口0忙的时候等待, 不忙的时候再发送传进来的字符
68         // Wait when serial port 0 is busy, and send the incoming characters when it is not busy
69         while( DL_UART_isBusy(UART_0_INST) == true );
70         //发送字符串首地址中的字符, 并且在发送完成之后首地址自增
71         // Send the characters in the first address of the string, and increment the first address after sending.
72         DL_UART_Main_transmitData(UART_0_INST, *str++);
73     }
74 }

```

The `uart0_send_string` function sends a string through UART serial port. It iterates through each character with `while (*str != 0)` until the string ends (`*str == 0`). Before sending each character, it checks if UART is busy via `DL_UART_isBusy(UART_0_INST)`. Data can only be sent when UART is idle. It sends the current character via `DL_UART_Main_transmitData(UART_0_INST, *str++)` and increments `str` to point to the next character.

```

43 //读取ADC的数据 Read ADC data
44 unsigned int adc_getValue(unsigned int number)
45 {
46     unsigned int gAdcResult = 0;
47     unsigned char i = 0;
48
49     //采集多次累加 Collect multiple times and accumulate
50     for( i = 0; i < number; i++ )
51     {
52         gAdcResult += ADC_VALUE[i];
53     }
54     //均值滤波 Mean Filter
55     gAdcResult /= number;
56
57     return gAdcResult;
58 }

```

This code reads ADC (analog-to-digital converter) data. It triggers ADC conversion and waits for conversion to complete, then gets the ADC sample result and returns it.

```

9   int main(void)
10  {
11      char output_buff[50] = {0};
12      unsigned int adc_value = 0;
13      float voltage_value = 0;
14
15      SYSCFG_DL_init();
16
17      //设置DMA搬运的起始地址 Set the starting address of DMA transfer
18      DL_DMA_setSrcAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC0->ULLMEM.MEMRES[0]);
19      //设置DMA搬运的目的地址 Set the destination address of DMA transfer
20      DL_DMA_setDestAddr(DMA, DMA_CH0_CHAN_ID, (uint32_t) &ADC_VALUE[0]);
21      //开启DMA Enable DMA
22      DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
23      //开启ADC转换 Start ADC conversion
24      DL_ADC12_startConversion(ADC_VOLTAGE_INST);
25
26      uart0_send_string("adc+dma Demo start\r\n");
27      while (1)
28      {
29          //获取ADC数据 Get ADC data
30          adc_value = adc_getValue(10);
31          sprintf(output_buff, "adc value:%d\r\n", adc_value);
32          uart0_send_string(output_buff);
33
34          //将ADC采集的数据换算为电压 Convert the data collected by ADC into voltage
35          voltage_value = adc_value/4095.0*3.3;
36          sprintf(output_buff, "voltage value:%.2f\r\n", voltage_value);
37          uart0_send_string(output_buff);
38
39          delay_cycles(32000000);
40      }
41  }

```

This code's functionality is to use **ADC** (analog-to-digital converter) and **DMA** (Direct Memory Access) combined to acquire analog signals and output results through serial port. DMA transfers ADC data directly to memory without manual reading each time, and periodically outputs ADC sample values and calculated voltage values through serial port.

5. Experimental Phenomena

After the program is downloaded, after configuring the serial assistant as shown in the figure below, adjust the potentiometer knob to output voltage to the PA27 pin. After ADC collects and loads data into ADC result register 0, DMA is triggered. DMA will transfer data to the memory variable ADC_VALUE. You can get data by directly calling ADC_VALUE.

